

National Taiwan Normal University
CSIE Computer Programming I

Instructor: Po-Wen Chi
Due Date: 2024.12.17 PM 11:59

Assignment 5

Policies:

- Zero tolerance for late submission.
- Please pack all your submissions in one zip file. **RAR is not allowed!!**
- For convenience, your executable programs must be named following the rule hw**XXYY**, where the red part is the homework number and the blue part is the problem number. For example, **hw0102** is the executable program for homework #1 problem 2.
- I only accept **PDF** or **TEXT**. MS Word is not allowed.
- **Do not forget your Makefile. For convenience, each assignment needs only one Makefile.**
- Please provide a README file. The README file should have at least the following information:
 - Your student ID and your name.
 - How to build your code.
 - How to execute your built programs.
 - Anything that you want to notify our TAs.
- **DO NOT BE A COPYCAT!!** You will get ZERO if you are caught.

5.1 Point Mirroring (20 pts)

Given a point P in 2-D plane and a line, please implement a program to find the image of that point Q formed due to the mirror. Figure 5.1 is an example.

You need to implement the following functions:

```
1 // Use (x1,y1) and (x2,y2) to determine a line.
2 // If the input is not a valid line, use previous valid one.
3 void set_line( double x1, double y1, double x2, double y2 );
```

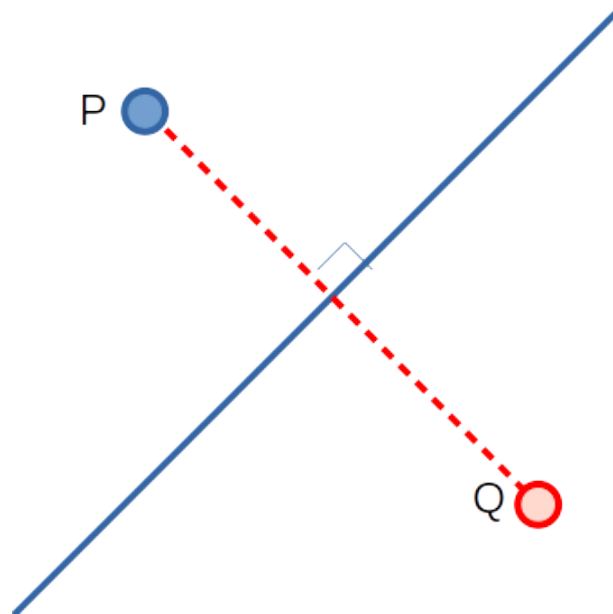


FIGURE 5.1: Point Mirroring.

```

4 // Q(c,d) is the mirror of P(a,b) according to the pre-
   determined line.
5 // If there is no valid line, return -1; otherwise, return 0.
6 int32_t get_mirror( double a, double b, double *c, double *d );

```

You should also prepare a header file called `mirror.h`. Our TAs will prepare `hw0501.c` which includes `mirror.h` and uses these functions. **Do not forget to make `hw0501.c` to `hw0501` in your Makefile.**

5.2 Gaussian Elimination (20 pts)

Given a matrix $\mathbf{A}$ and a vector $\mathbf{y}$, please find a vector $\mathbf{x}$ so that $\mathbf{Ax} = \mathbf{y}$. If it is unfamiliar to you, do not worry. I will give you an example as follows. Given

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 4 \\ 1 & 3 & 9 \end{bmatrix}, \mathbf{y} = \begin{bmatrix} 6 \\ 17 \\ 34 \end{bmatrix},$$

You can get $\mathbf{x}$ is

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \text{ because } \mathbf{Ax} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 4 \\ 1 & 3 & 9 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 6 \\ 17 \\ 34 \end{bmatrix}.$$

Actually, this problem is equal to solve the following problem.

$$\begin{cases} x_1 + x_2 + x_3 = 6 \\ x_1 + 2x_2 + 4x_3 = 17 \\ x_1 + 3x_2 + 9x_3 = 34 \end{cases}$$

This time, I want you to develop a function to solve this problem.

```

1 // I promise that A is an nxn matrix (2d array).
2 // However, you need to check if the inputs are valid.
3 // You should malloc for x.
4 // Return -1 if the inputs are invalid. *px should be NULL.
5 // Return 0 if there is no solution. *px should be NULL.
6 // Return 1 if there is only one solution. x should be the
  solution
7 // Return 2 if there are more than one solutions. x should be
  one possible solution.
8 int32_t gaussian_elimination( int32_t n, int32_t *pA, int32_t *
  py, int32_t **px );

```

You need to implement the following functions with a header file called **myge.h**. Our TAs will prepare **hw0502.c** which includes **myge.h** and uses these functions. **Do not forget to make [hw0502.c](#) to [hw0502](#) in your Makefile.** For your simplicity, I promise that all answers are 32-bits integers.

5.3 General Endian Converter (20 pts)

In the class, I have taught you what endian is. This time, I want you to develop an endian converter function and equality test function.

```

1 // Input:
2 //   uint8_t *pSrc: The number that should be converted.
3 //   uint32_t size: src size
4 //   uint8_t **ppDst: Converted result
5 // Return:
6 //   -1 for invalid inputs; otherwise, 0
7 int32_t endian_convert( uint8_t *pSrc, uint32_t size, uint8_t
  **ppDst );
8
9 // Input:
10 //   uint8_t *pNum1, *pNum2: The numbers.
11 //   uint32_t size1, size2: src size of Numbers.
12 //   uint8_t type1, type2: 0: little endian, 1: big endian;
   others: invalid
13 // Return:
14 //   -2 for invalid inputs; Num1 < Num2: -1; Num1 = Num2: 0;
   Num1 > Num2: 1;
15 int32_t endian_equality_test( uint8_t *pNum1, uint32_t size1,
  uint8_t type1,
16                               uint8_t *pNum2, uint32_t size2,
  uint8_t type2 );

```

You need to implement the following functions with a header file called **endian.h**. Our TAs will prepare hw0503.c which includes endian.h and uses these functions. **Do not forget to make hw0503.c to hw0503 in your Makefile.**

5.4 DHCP (20 pts)

The Dynamic Host Configuration Protocol (DHCP) provides a framework for automatic configuration of IP hosts. The document "DHCP Options and BOOTP Vendor Information Extensions" [RFC2132] describes options for DHCP. DHCP option field is as regular TLV field, that is, **type, length, and value**. The first octet is the option code, the second octet is the number of following octets and the remaining octets are code dependent. You can see the following link for reference.

https://en.wikipedia.org/wiki/Dynamic_Host_Configuration_Protocol#Options

This time, I want you to develop a DHCP option parser. The parsing function is defined as follows.

```
1 // This function is used to print one TLV of data at a time.
2 // If the user wants to print next TLV, it should call
   dhcpParse( NULL, 0 );
3 // Print format:
4 // Code: XX
5 // Name: YY
6 // Size: ZZ
7 // Data: AAAAAA
8 // A print example is as follows.
9 // Code: 5
10 // Name: Name server
11 // Size: 4
12 // Data: 8.8.8.8, 168.95.1.1
13 // Input:
14 //     uint8_t *pData: data
15 //     uint32_t size: the size of data
16 // Return:
17 //     For invalid inputs or no TLVs left, return -1; The end of
       TLV, return 0; Otherwise, return 1.
18 int32_t dhcpParse( uint8_t *pData, uint32_t size );
```

For your simplicity, you only need to support code **0-12, 19, 20, 255**. The data print formats are defined as follows.

- 1,3-11: XX.XX.XX.XX. XX is an integer between 0 and 255 for each byte. Do not forget the dots between each integers.
- 12: Print each byte with %c.

- 19,20: 0 or 1.

You should also prepare a header file called `dhcp.h`. Our TAs will prepare `hw0504.c` which includes `dhcp.h` and uses these functions. **Do not forget to make `hw0504.c` to `hw0504` in your Makefile.**

5.5 Turing Machines exists!!! (20 pts)

Panda is a CSIE student, he believes that Turing Machines exists!

But he is bad at programming, so he wants you to solve a Turing Machine Problem for him.

Here is the problem statement: There are 2 tapes (imagine them as 2 memories) and 2 pointers pointing to the start of the tape.

The first tape consists of 2 arrays `x` and `y` split by `-1`, which looks like `[-1, x1, x2, ..., xn, -1, y1, y2, ..., ym, -1]`.

The second tape is structured like `[-1, -1, ...]`

Notice that `-1` represents a "split mark" here for simplicity. Meaning that if you put something on tape 2 and move right, you will point to another `"-1"`. (Which means that you don't need to worry about allocating undeclared memory in tapes.)

Now, your task is to make convert the second tape into the following structure: `[-1, x0y0, x0y1, ..., x0ym, ..., xn, ym]`

For example:

Tape 1 has the following data: `"-1, 0, 1, 0, -1, 1, 2, 3, 4, -1"`. (`x="010"`, `y="1234"`)

Tape 2 should be converted into `"-1, 0, 0, 0, 0, 1, 2, 3, 4, 0, 0, 0, 0, -1"` after going through the Turing machine.

You will need to implement the following functions:

```

1  \\ ptape1 points to x1, and ptape2 points to the first position
   of tape 2 by default.
2  \\ I promise that the inputs are all valid.
3  \\ You are NOT allowed to declare any variable.
4  \\ x is in {0,1}, y is in [0,9].
5  \\ tape 1 should restore to its original state after running.
6  \\ pointers should point to the beginning of tapes. (not the
   first -1, it is a "wall" here)
7  void TuringMachine(uint8_t* ptape1, uint8_t* ptape2);

```

Hint:

1. You can use `ptape++` or `ptape--` to move the pointer right or left.
2. You can "mark" a location in tape 1 by adding 10 and "unmark" by subtracting 10.
3. It is recommended for you to create something like an automata.

tape1	-1	0	1	0	-1	1	2	3	4	-1				
tape2	-1	-1	...											
tape1	-1	0	1	0	-1	1	2	3	4	-1				
tape2	-1	0	0	0	0	1	2	3	4	0	0	0	0	-1

FIGURE 5.2: example for tape

5.6 Bonus: What is Wrong? (5 pts)

There is a programming problem: Please write a program to ask the user to input 5 16-bits integers and print these integers in the reverse order. Alice and Bob write their own codes and the execution results are as follows.

```

1 // Alice Code
2 #include <stdio.h>
3 #include <stdint.h>
4
5 int main()
6 {
7     int16_t array[10] = {0};
8
9     for( int32_t i = 0; i < 5 ; i++ )
10    {
11        scanf( "%d", &array[i] );
12    }
13
14    for( int32_t i = 5; i > 0; i-- )
15    {
16        printf( "%d ", array[i-1] );
17    }
18
19    printf( "\n" );
20
21    return 0;
22 }
```

```

1 $ ./a.out
2 1
3 2
4 3
5 4
6 5
7 5 4 3 2 1
```

```

1 // Bob Codes
2 #include <stdio.h>
3 #include <stdint.h>
4
5 int main()
6 {
```

```

7     int16_t array[10] = {0};
8
9     for( int32_t i = 5; i > 0 ; i-- )
10    {
11        scanf( "%d", &array[i-1] );
12    }
13
14    for( int32_t i = 0; i < 5; i++ )
15    {
16        printf( "%d ", array[i] );
17    }
18
19    printf( "\n" );
20
21    return 0;
22 }

```

```

1 $ ./a.out
2 1
3 2
4 3
5 4
6 5
7 5 0 0 0 0

```

What is wrong with Bob's code? Please explain the result. **Note that I do not want you to correct their codes but I want you to describe the reason.** Besides, why the array size is ten? What will happen if the size is five?